

CS307 Project2 数据库内核实现讲解版 README

这份 README 的目标不是简单告诉你“项目能跑”，而是帮你在答辩前真正知道：

- 一个 SQL 从输入到输出经过哪些类。
- 每个包负责什么。
- 每个关键类和关键方法实现了什么。
- 老师问到 **SeqScan**、事务、索引、Join、**GROUP BY** 时该从哪里讲起。

当前项目是一个教学型 Java 数据库内核，不是完整 MySQL/PostgreSQL。它实现了课程 Project2 需要的核心链路：SQL 解析、逻辑算子、物理算子、记录管理、缓冲池、磁盘页、B+ Tree 内存索引、事务快照和命令行展示。

一句话总览

项目主执行链路是：

```
DBEntry 读入 SQL
-> LogicalPlanner 把 SQL 解析成 LogicalOperator
-> PhysicalPlanner 把 LogicalOperator 转成 PhysicalOperator
-> PhysicalOperator 按 Begin / hasNext / Next / Current / Close 执行
-> Tuple 返回结果行
-> RecordFileHandle / BufferPool / DiskManager 完成底层读写
```

你答辩时最应该抓住这条线。所有功能都能放进这条线里解释。

当前支持的 SQL 功能

基础功能：

- **CREATE TABLE**
- **DROP TABLE**
- **SHOW TABLES**
- **DESC t / DESCRIBE t**
- **EXPLAIN SELECT ...**
- **INSERT**
- **UPDATE**
- **DELETE**
- **SELECT ***
- **SELECT t.id, t.name**
- **WHERE** 条件过滤
- **AND / OR**
- **= / > / >= / < / <= / != / <>**
- **COUNT(*)**
- 简单 **JOIN**

高级功能：

- MAX()
- MIN()
- GROUP BY
- ORDER BY ASC/DESC
- ORDER BY 投影别名
- IN
- NOT IN
- EXISTS
- NOT EXISTS
- 简单相关子查询
- CREATE INDEX
- DROP INDEX
- PRINT INDEX
- 内存 B+ Tree 索引
- BEGIN
- COMMIT
- ROLLBACK
- SAVEPOINT
- ROLLBACK TO SAVEPOINT
- RELEASE SAVEPOINT
- 空表上的 ALTER TABLE ADD COLUMN
- 空表上的 ALTER TABLE DROP COLUMN

已知边界：

- ALTER TABLE 只支持空表，不支持非空表数据迁移。
- B+ Tree 节点本身是内存结构，不把树节点持久化到磁盘；索引元数据会保存，运行时可重建。
- 优化器是课程项目级优化器，不是成本优化器。
- 复杂 SQL 如窗口函数、外键、唯一约束、多列索引、完整嵌套查询语义没有实现。

运行和测试

项目代码主体在：

```
engine-project-master/
```

进入工程目录：

```
cd D:\数据库原理\cs307-project2\engine-project-master
```

运行全量测试：

```
& ..\tools\apache-maven-3.9.11\bin\mvn.cmd "-Dmaven.repo.local=..\m2\repository"
test
```

当前全量测试结果：

```
Tests run: 115, Failures: 0, Errors: 0
```

运行入口类：

```
src/main/java/edu/sustech/cs307/DBEntry.java
```

测试重点：

```
src/test/java/storage/LRUReplacerTest.java
src/test/java/storage/ClockReplacerTest.java
src/test/java/system/Task2IntegrationTest.java
src/test/java/system/IndexIntegrationTest.java
src/test/java/system/AdvancedQueryIntegrationTest.java
src/test/java/system/TransactionManagerTest.java
src/test/java/record/RecordFileHandleTest.java
```

包结构总览

edu.sustech.cs307	
├ DBEntry.java	命令行入口
├ exception	自定义异常
├ value	数据值和值比较
├ meta	表元数据、列元数据、索引元数据
├ optimizer	SQL 到逻辑/物理计划
├ logicalOperator	逻辑算子
├ logicalOperator.ddl	DDL 执行器
├ physicalOperator	物理算子，真正执行查询和修改
├ tuple	查询执行过程中流动的一行数据
├ record	记录文件、页内记录、RID、Bitmap
├ storage	磁盘页、缓冲池、磁盘管理
├ storage.replacer	LRU / Clock 页面替换算法
├ system	DBManager、RecordManager、TransactionManager
└ index	B+ Tree 索引

下面按执行链路和包逐个讲。

1. 命令行入口：DBEntry

文件：

```
engine-project-master/src/main/java/edu/sustech/cs307/DBEntry.java
```

核心作用：

- 初始化数据库运行环境。
- 循环读取用户输入 SQL。
- 调用 planner 生成执行计划。
- 执行物理算子。
- 把结果打印成表格。

关键方法：

```
main(String[] args)
```

这是程序入口。

它做了几件事：

1. 配置 tinylog 日志格式。
2. 创建 `DiskManager`。
3. 创建 `BufferPool`。
4. 创建 `RecordManager`。
5. 创建 `MetaManager`。
6. 创建 `DBManager`。
7. 用 `JLine` 循环读取 SQL。
8. 调用：

```
LogicalOperator operator = LogicalPlanner.resolveAndPlan(dbManager, sql);
PhysicalOperator physicalOperator = PhysicalPlanner.generateOperator(dbManager,
operator);
```

9. 对物理算子执行：

```
physicalOperator.Begin();
while (physicalOperator.hasNext()) {
    physicalOperator.Next();
    Tuple tuple = physicalOperator.Current();
    ...
}
physicalOperator.Close();
```

这就是典型的火山模型，也就是数据库执行器常见的 iterator 模型。

```
getHeaderString(ArrayList<ColumnMeta> columnMetas)
```

把输出 schema 转成表头，例如：

```
|      t.id      |      t.name      |
```

`getRecordString(Tuple tuple)`

调用 `tuple.getValues()` 得到一行里的所有值，再打印。

答辩说法：

`DBEntry` 不负责理解 SQL，它只负责输入输出和串起执行流程。SQL 真正变成计划是在 `LogicalPlanner` 和 `PhysicalPlanner`。

2. SQL 解析和逻辑计划：optimizer.LogicalPlanner

文件：

```
engine-project-  
master/src/main/java/edu/sustech/cs307/optimizer/LogicalPlanner.java
```

这是 SQL 入口类，最重要。

`resolveAndPlan(DBManager dbManager, String sql)`

所有 SQL 都先进入这个方法。

执行顺序：

1. 空 SQL 直接返回 `null`。
2. `handleManualTransactionCommand` 先识别事务命令。
3. `handleManualDdlCommand` 识别手写 DDL，比如 `SHOW TABLES`、`DROP TABLE`、`CREATE INDEX`、`ALTER TABLE`。
4. 其他 SQL 交给 `JSqlParser` 解析。
5. 根据解析结果类型分发：

```
Select      -> handleSelect  
Insert      -> handleInsert  
Update      -> handleUpdate  
Delete      -> handleDelete  
CreateTable -> CreateTableExecutor.execute()  
Explain     -> ExplainExecutor.execute()  
Show        -> ShowDatabaseExecutor.execute()
```

为什么有一些命令用正则手写？

因为教学框架和 `JSqlParser` 对一些命令支持不完全，或者我们希望控制语法边界，例如：

- BEGIN
- ROLLBACK TO SAVEPOINT
- CREATE INDEX
- PRINT INDEX
- ALTER TABLE ADD COLUMN

这些都通过正则提前截获，然后直接调用 `DBManager`。

`handleSelect(DBManager dbManager, Select selectStmt)`

把 `SELECT` 语句转成逻辑算子树。

普通查询的逻辑计划大概是：

```
LogicalTableScanOperator
-> LogicalJoinOperator      如果有 JOIN
-> LogicalFilterOperator    如果有 WHERE
-> LogicalCountOperator     如果是 COUNT(*)
-> LogicalProjectOperator   普通投影
```

例子：

```
select t.id, t.name from t where t.age > 18;
```

逻辑树：

```
Project
├─ Filter(t.age > 18)
│   └─ TableScan(t)
```

`requiresAdvancedSelect(PlainSelect plainSelect)`

判断这个 `SELECT` 要不要走高级查询算子。

以下情况会走 `LogicalAdvancedSelectOperator`：

- 有 `GROUP BY`
- 有 `HAVING`
- 有 `ORDER BY`
- 有 `MAX()`
- 有 `MIN()`
- `WHERE` 里有 `IN`
- `WHERE` 里有 `EXISTS`

也就是说，普通查询保持原来的流式执行，高级查询用物化执行。

isCountSelect(PlainSelect plainSelect)

识别：

```
select count(*) from t;
```

如果是简单 `COUNT(*)`，逻辑层生成 `LogicalCountOperator`。

handleManualTransactionCommand(DBManager dbManager, String sql)

处理事务命令：

```
begin;  
start transaction;  
commit;  
rollback;  
savepoint s1;  
rollback to savepoint s1;  
release savepoint s1;
```

这些命令不会生成物理算子，直接调用 `TransactionManager`，所以返回 `null`。

handleManualDdlCommand(DBManager dbManager, String sql)

处理 DDL 和索引命令：

```
show tables;  
desc t;  
drop table t;  
create index idx on t(age);  
drop index idx;  
print index idx;  
alter table t add column c int;  
alter table t drop column c;  
explain select ...;
```

这些命令大多数直接调用 `DBManager`，不会进入普通物理算子流程。

答辩说法：

`LogicalPlanner` 的任务是把 SQL 的语义转成逻辑算子，不直接读写数据。它只决定“要做什么”，不决定“怎么具体执行”。

3. 逻辑算子：logicalOperator

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/logicalOperator
```

逻辑算子只描述操作，不真正执行。

LogicalOperator

所有逻辑算子的父类。

字段：

```
protected List<LogicalOperator> children;
```

作用：

- 保存子节点。
- 提供 `getChild()` 和 `getChildren()`。
- 每个子类实现 `toString()`，用于 `EXPLAIN`。

LogicalTableScanOperator

表示：

```
from t
```

关键字段：

- `tableName`

后面会被 `PhysicalPlanner` 转成 `SeqScanOperator`。

LogicalFilterOperator

表示：

```
where t.age > 18
```

关键字段：

- `child`
- `whereExpr`

`whereExpr` 是 `JSqlParser` 的 `Expression`。

LogicalProjectOperator

表示：

```
select t.id, t.name
```

关键方法：

```
getOutputSchema()
```

这个方法把 `SelectItem` 转成项目里自己的 `TabCol` 列描述。

支持：

- `SELECT *`
- `SELECT t.id`
- `SELECT id`

LogicalJoinOperator

表示：

```
t join d on t.id = d.student_id
```

关键字段：

- 左输入
- 右输入
- join 条件
- join 深度

后面转成 `NestedLoopJoinOperator`。

LogicalInsertOperator

保存 insert 的表名、列名和值表达式。

真正的值类型检查在 `PhysicalPlanner.handleInsert()` 里做。

LogicalUpdateOperator

保存 update 的表名、更新列、更新表达式和 where 条件。

LogicalDeleteOperator

保存 delete 的表名和 where 条件。

LogicalCountOperator

表示：

```
select count(*) ...
```

LogicalAdvancedSelectOperator

表示高级查询：

```
select age, count(*), max(gpa) from t group by age order by age;
```

它保存完整的 `PlainSelect`，后面直接交给 `AdvancedSelectOperator` 物化执行。

4. 物理计划：optimizer.PhysicalPlanner

文件：

```
engine-project-  
master/src/main/java/edu/sustech/cs307/optimizer/PhysicalPlanner.java
```

逻辑计划只说明“要做什么”，物理计划说明“用哪个算子做”。

`generateOperator(DBManager dbManager, LogicalOperator logicalOp)`

这是物理计划入口。

映射关系：

LogicalTableScanOperator	-> SeqScanOperator
LogicalFilterOperator	-> FilterOperator 或 IndexScanOperator +
FilterOperator	
LogicalJoinOperator	-> NestedLoopJoinOperator
LogicalProjectOperator	-> ProjectOperator
LogicalInsertOperator	-> InsertOperator
LogicalUpdateOperator	-> UpdateOperator
LogicalDeleteOperator	-> DeleteOperator
LogicalCountOperator	-> CountOperator
LogicalAdvancedSelectOperator	-> AdvancedSelectOperator

handleFilter(...)

这里有一个简单优化器逻辑。

如果发现：

```
select ... from t where t.age >= 20;
```

并且 `age` 上有索引，则调用：

```
dbManager.lookupIndex(tableName, columnName, operator, value)
```

拿到满足条件的 RID 列表，然后使用：

```
new IndexScanOperator(dbManager, tableName, lookup.rids)
```

最后外面再包一层 `FilterOperator`，保证即使索引只利用了部分条件，完整 `where` 条件仍然会再判断一遍。

答辩说法：

这里不是完整成本优化器，但实现了基于索引可用性的简单 plan 选择：有可用索引用 `IndexScan`，没有就 `SeqScan`。

handleInsert(...)

把逻辑插入转成物理插入。

做的事情：

1. 读取 `TableMeta`。
2. 检查列数量是否匹配。
3. 检查列名是否存在。
4. 检查值类型是否和列类型匹配。
5. 把 `JSqlParser` 的常量表达式转成 `Value`。
6. 创建 `InsertOperator`。

parseValue(...)

把 SQL 值转成内部值：

```
StringValue -> ValueType.CHAR  
DoubleValue -> ValueType.FLOAT  
LongValue   -> ValueType.INTEGER
```

注意：字符串最长按 `Value.CHAR_SIZE` 截断。

findIndexedLookup(...)

递归检查 `where` 条件里是否有可以用索引的子条件。

支持：

```
where age = 20
where age > 20
where age >= 20
where age < 20
where age <= 20
where age >= 20 and id <= 10
```

如果是 **AND**，会尝试从左右子表达式找一个可以使用索引的条件。

5. 物理算子接口：physicalOperator.PhysicalOperator

文件：

```
engine-project-
master/src/main/java/edu/sustech/cs307/physicalOperator/PhysicalOperator.java
```

所有物理算子都实现同一个接口：

```
boolean hasNext() throws DBException;
void Begin() throws DBException;
void Next() throws DBException;
Tuple Current();
void Close();
ArrayList<ColumnMeta> outputSchema();
```

这是火山模型。

含义：

- **Begin()**：打开算子，准备执行。
- **hasNext()**：判断还有没有下一行。
- **Next()**：移动到下一行。
- **Current()**：返回当前行。
- **Close()**：关闭算子，释放资源。
- **outputSchema()**：返回输出列结构。

答辩说法：

每个算子都像一个迭代器。上层算子不断向下层算子要数据，下层算子一行一行产生结果。

6. 普通查询物理算子：physicalOperator

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/physicalOperator
```

SeqScanOperator

最基础、最重要的扫描算子。

负责：

- 打开表的数据文件。
- 从第一个数据页开始扫描。
- 在每页内按 slot 扫描。
- 使用 bitmap 判断 slot 是否有记录。
- 把记录包装成 `TableTuple`。

关键方法：

Begin()

打开 record 文件：

```
fileHandle = dbManager.getRecordManager().OpenFile(tableName);
```

初始化：

```
currentPageNum = 0  
currentSlotNum = 0  
totalPages = numberOfPages - 2
```

注意页号语义：

- 物理 page 0 是文件头。
- RID 里的 `pageNum` 是逻辑数据页号，从 0 开始。
- 所以逻辑数据页 0 对应物理 page 1。

hasNext()

核心扫描逻辑：

1. 当前页没扫完就继续看 slot。
2. 用 `BitMap.isSet(pageHandle.bitmap, currentSlotNum)` 判断 slot 有没有记录。
3. 如果有记录，返回 `true`。
4. 当前页扫完后跳到下一页。

Next()

找到下一条记录后：

```
currentRid = new RID(currentPageNum, currentSlotNum);
currentRecord = fileHandle.GetRecord(currentRid);
```

然后 slot 往后移动。

Current()

把当前记录封装成：

```
new TableTuple(tableName, tableMeta, currentRecord, new RID(currentRid))
```

答辩说法：

SeqScan 不理解 SQL 条件，它只负责把表里所有有效记录按顺序吐出来。WHERE 是 **FilterOperator** 做的。

FilterOperator

负责：

```
where ...
```

它包装一个 child operator，每次从 child 取 tuple，然后调用：

```
tuple.eval_expr(whereExpr)
```

只有条件为 true 的 tuple 才会输出。

ProjectOperator

负责：

```
select t.id, t.name
```

它包装 child operator，然后把 child 的 tuple 包成 **ProjectTuple**。

ProjectTuple 只暴露投影列。

NestedLoopJoinOperator

负责简单 nested loop join。

执行思想：

1. `Begin()` 时读取右表所有 tuple 到内存列表。
2. 左表每拿一行，就和右表所有行组合。
3. 用 join 条件判断是否匹配。
4. 匹配后输出 `JoinTuple`。

适合教学展示，但不是高性能 join。

CountOperator

负责：

```
select count(*) from t where ...
```

逻辑：

1. `Begin()` 时遍历 child 的所有 tuple。
2. 计数。
3. 最后输出一行一列，列名是 `count`。

如果有 where，则 where 会在 child 的 `FilterOperator` 里先过滤。

InsertOperator

负责真正插入数据。

关键方法：

Begin()

1. 打开数据文件：

```
dbManager.getRecordManager().OpenFile(data_file)
```

2. 把 `Value` 序列化进 `ByteBuf`。
3. 调用：

```
fileHandle.InsertRecord(buffer)
```

4. 拿到 RID 后更新索引：

```
dbManager.addToIndexes(...)
```

Current()

返回插入行数，例如：

```
numberOfInsertRows = 3
```

UpdateOperator

负责：

```
update t set age = 20 where id = 1;
```

逻辑：

1. 从 child scanner 拿满足 where 的 tuple。
2. 找到要更新的列。
3. 重新序列化整条记录。
4. 调用 `RecordFileHandle.UpdateRecord(...)`。
5. 调用 `dbManager.updateRecordIndexes(...)` 同步更新索引。

当前边界：

- 主要支持单列更新。

DeleteOperator

负责：

```
delete from t where id = 1;
```

逻辑：

1. 遍历 child。
2. 判断 where。
3. 删除前先从索引里移除 RID。
4. 调用 `RecordFileHandle.DeleteRecord(...)`。
5. 返回删除行数。

IndexScanOperator

当 `PhysicalPlanner` 发现某个 where 条件能用索引时，会生成这个算子。

它不是扫全表，而是：

1. 已经拿到一批 RID。
2. 按 RID 直接取记录。
3. 输出 `TableTuple`。

这样避免扫描整个表。

InMemoryIndexScanOperator

兼容内存索引扫描路径，目前项目主要通过 `IndexScanOperator` 使用 RID 列表。

AdvancedSelectOperator

这是高级 SELECT 的核心。

文件：

```
engine-project-  
master/src/main/java/edu/sustech/cs307/physicalOperator/AdvancedSelectOperator.java
```

支持：

- `MAX`
- `MIN`
- `COUNT` 分组
- `GROUP BY`
- `ORDER BY`
- `IN`
- `NOT IN`
- `EXISTS`
- `NOT EXISTS`
- 简单相关子查询

为什么它和普通查询分开？

普通查询可以流式执行，一行一行过即可。`GROUP BY`、`ORDER BY`、聚合、子查询更适合先把中间结果物化到内存，再处理。

关键方法：

Begin()

执行整个高级查询：

```
results = executePlainSelect(plainSelect, null, true);
```

把最终结果放在 `results` 列表里，后面 `hasNext/Next/Current` 只是从列表里迭代。

`executePlainSelect(...)`

高级查询主流程：

```
materializeSourceRows
-> WHERE filter
-> aggregate/group 或 project
-> ORDER BY
-> 返回 MaterializedTuple 列表
```

`materializeSourceRows(...)`

把 `from` 和 `join` 的输入表都读成内存 tuple。

如果有 `join`：

```
左表 rows x 右表 rows
-> combine
-> matchesJoin
```

`evaluateCondition(...)`

判断布尔条件。

支持：

- 括号
- `NOT`
- `AND`
- `OR`
- 二元比较
- `IN`
- `EXISTS`

普通 `Tuple.eval_expr` 只处理基础 `where`；高级 `where` 在这里处理。

`evaluateInExpression(...)`

处理：

```
where id in (1, 3, 5)
where id in (select student_id from d)
where id not in (...)
```

逻辑：

1. 先算左边表达式的值。
2. 如果右边是常量列表，就逐个比较。
3. 如果右边是子查询，就执行子查询，取第一列比较。
4. 如果是 **NOT IN**，最后取反。

executeSelectExpression(...)

执行子查询。

比如：

```
where exists (select d.student_id from d where d.student_id = t.id)
```

如果子查询里引用了外层表的列，方法会把 outer tuple 合并进子查询执行环境，实现简单相关子查询。

aggregateRows(...)

处理 **GROUP BY** 和聚合。

流程：

1. 根据 group by 表达式计算 group key。
2. 用 **LinkedHashMap** 把行分组。
3. 对每个组计算 select item。
4. 输出一行 **MaterializedTuple**。

computeAggregate(...)

支持：

```
count(*)
count(col)
max(col)
min(col)
```

max/min 比较使用 **ValueComparer.compare(...)**。

sortRows(...)

处理：

```
order by age
order by age desc
order by sid desc
```

这里也支持投影别名：

```
select t.id as sid from t order by sid desc;
```

buildOutputSchema(...)

构造结果列元数据，让最终打印表头知道列名和类型。

答辩说法：

`AdvancedSelectOperator` 采用物化执行策略，先把查询涉及的行读成 `MaterializedTuple`，然后统一做 where、聚合、排序和投影。这样实现简单，适合课程项目的高级 SQL。

7. Tuple：执行过程中流动的一行数据

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/tuple
```

Tuple

抽象父类。

关键方法：

```
Value getValue(TabCol tabCol)
TabCol[] getTupleSchema()
Value[] getValues()
boolean eval_expr(Expression expr)
Value evaluateExpression(Expression expr)
```

`eval_expr` 用于基础 where 判断。

支持：

- AND
- OR
- 二元比较
- 常量
- 列引用

TableTuple

表示从真实表记录里读出的一行。

内部包含：

- 表名
- 表元数据 `TableMeta`
- 底层 `Record`
- `RID`

`getValue(TabCol tabCol)` 会根据列 `offset` 和 `len` 从 `Record` 里切出字节，然后转成 `Value`。

ProjectTuple

表示投影之后的一行。

比如 child tuple 有：

```
t.id, t.name, t.age, t.gpa
```

查询只要：

```
select t.id, t.name
```

那么 `ProjectTuple` 的 `schema` 只包含这两列。

JoinTuple

表示 join 后的一行。

内部保存：

- left tuple
- right tuple
- join 后 schema

`getValue` 先去左边找，找不到再去右边找。

TempTuple

临时输出 tuple。

用于：

- insert 返回插入行数
- update 返回更新行数
- delete 返回删除行数
- count 返回计数

它一般不支持按列名取值。

MaterializedTuple

高级查询使用的物化 tuple。

特点：

- 同时保存 schema 和 values。
- 支持按列名取值。
- 适合 **ORDER BY**、**GROUP BY**、子查询这种需要反复读取值的场景。

8. 元数据：meta

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/meta
```

元数据就是“表长什么样”，不是表里的真实数据。

ColumnMeta

描述一列：

```
String tableName;  
String name;  
ValueType type;  
int len;  
int offset;
```

字段解释：

- **tableName**：属于哪张表。
- **name**：列名。
- **type**：列类型。
- **len**：序列化后占多少字节。
- **offset**：这列在 record 里的起始偏移。

例子：

```
create table t(id int, name varchar, age int);
```

可能对应：

```
id    offset 0    len 8
name offset 8    len 64
age   offset 72   len 8
```

TableMeta

描述一张表。

字段：

- `tableName`
- `columns_list`
- `columns`
- `indexes`
- `indexColumns`

关键方法：

`addColumn(ColumnMeta column)`

给表增加列元数据。

用于空表 `ALTER TABLE ADD COLUMN`。

`dropColumn(String columnName)`

删除列元数据。

如果这个列上有索引，也会移除索引元数据。

`addIndex(String indexName, String columnName)`

记录索引元数据：

```
indexName -> BTREE
indexName -> columnName
```

`dropIndex(String indexName)`

删除索引元数据。

`getIndexNameOnColumn(String columnName)`

给定列名，查找该列上是否有索引。

`PhysicalPlanner` 判断能不能走索引时会用到。

MetaManager

管理所有表的元数据，并负责 JSON 持久化。

关键方法：

createTable(TableMeta tableMeta)

创建表元数据并保存到 JSON。

dropTable(String tableName)

删除表元数据并保存。

addColumnInTable(...)

给某张表增加列，并保存 JSON。

dropColumnInTable(...)

删除某张表的列，并保存 JSON。

getTable(String tableName)

根据表名获取 TableMeta。

findTableByIndexName(String indexName)

根据索引名反查属于哪张表。

saveToJson() / reloadFromJson()

持久化和重新加载元数据。

答辩说法：

元数据存在 JSON 里，真实数据存在表目录下的 data 文件里。元数据告诉系统每条 record 应该怎样解析。

9. DDL 执行器：logicalOperator.ddl

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/logicalOperator/ddl
```

CreateTableExecutor

负责执行：


```
create table t(id int, name varchar, age int, gpa double);
```

关键方法：

execute()

流程：

1. 读取表名。
2. 遍历 column definitions。
3. 把 SQL 类型转成内部类型：

```
char/varchar      -> ValueType.CHAR, len 64
int/integer       -> ValueType.INTEGER, len 8
float/double      -> ValueType.FLOAT, len 8
```

4. 为每列计算 offset。
5. 调用：

```
dbManager.createTable(table, colMapping);
```

ExplainExecutor

原框架 DDL 执行器之一。现在实际 **EXPLAIN** 主要在 `LogicalPlanner.handleManualDdlCommand` 中手写处理，会调用 `explainSql(...)` 打印逻辑计划。

ShowDatabaseExecutor

原框架 show 执行器之一。当前 **SHOW TABLES** 主要由 `DBManager.showTables()` 输出。

10. DBManager：系统总管

文件：

```
engine-project-master/src/main/java/edu/sustech/cs307/system/DBManager.java
```

这是各模块的门面类，很多 SQL 命令最终都会调它。

它持有：

```
MetaManager metaManager;
DiskManager diskManager;
BufferPool bufferPool;
```

```
RecordManager recordManager;  
TransactionManager transactionManager;  
Map<String, Map<String, BPlusTreeIndex>> runtimeIndexes;
```

表管理方法

showTables()

打印当前数据库所有表。

descTable(String table_name)

读取 TableMeta，打印列名和类型。

createTable(String table_name, ArrayList<ColumnMeta> columns)

流程：

1. 创建 TableMeta。
2. 保存元数据。
3. 创建表目录。
4. 计算 record size。
5. 调用 recordManager.CreateFile(tableName/data, record_size) 创建数据文件。

dropTable(String table_name)

流程：

1. 检查表是否存在。
2. flush 数据页。
3. 从 buffer pool 删除页。
4. 删除 record 文件。
5. 删除表目录。
6. 删除元数据。
7. 删除运行时索引。

事务方法

beginTransaction()

调用 transactionManager.beginTransaction()。

commitTransaction()

调用 transactionManager.commit()。

persistRuntimeState()

把 buffer pool、disk manager metadata、meta manager JSON 都写盘。

事务创建 snapshot 前和 commit 时会调用。

索引方法

createIndex(String indexName, String tableName, String columnName)

流程：

1. 在 TableMeta 里记录索引。
2. 保存 metadata JSON。
3. 调用 rebuildIndex(...) 扫表建立 B+ Tree。

dropIndex(...)

删除 metadata 里的索引信息，并从 runtime index map 移除。

lookupIndex(String tableName, String columnName, String operator, Value value)

根据条件查索引。

支持：

```
=  
>  
>=  
<  
<=
```

返回满足条件的 RID 列表。

addRecordToIndexes(...)

insert 后调用。

把新行的索引列值插入 B+ Tree。

removeRecordFromIndexes(...)

delete 前调用。

从 B+ Tree 删除对应 key/RID。

updateRecordIndexes(...)

update 时调用。

先删除旧 key/RID，再插入新 key/RID。

printIndex(String indexName)

打印 B+ Tree 节点结构。

`rebuildIndex(String tableName, String indexName)`

扫描整张表，对每一行：

1. 取出索引列的值。
2. 取出 RID。
3. 插入 B+ Tree。

ALTER TABLE 方法

`alterTableAddColumn(String tableName, String columnName, String dataType)`

支持：

```
alter table t add column age int;
alter table t add name varchar;
```

限制：

- 只支持空表。

为什么只支持空表？

因为已有 record 的二进制布局已经固定。非空表加列需要重写所有记录，否则旧记录长度和新 schema 对不上。

流程：

1. 检查表存在。
2. 检查表为空。
3. 计算新列 offset。
4. 增加元数据。
5. 重建空 data 文件。

`alterTableDropColumn(String tableName, String columnName)`

支持空表删除列。

流程：

1. 检查表存在。
2. 检查表为空。
3. 检查不能删除到 0 列。
4. 删除列元数据。
5. 重新计算 offset。
6. 重建空 data 文件。

答辩说法：

DBManager 是系统门面。Planner 不直接操作磁盘，而是调用 DBManager；DBManager 再协调 MetaManager、RecordManager、BufferPool、DiskManager 和索引。

11. RecordManager 和 record 包：记录层

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/record  
engine-project-master/src/main/java/edu/sustech/cs307/system/RecordManager.java
```

这部分负责“表里的每一行如何存到页里”。

页号模型

这是答辩重点。

当前设计：

```
物理 page 0: RecordFileHeader 文件头页  
物理 page 1: 逻辑数据页 0  
物理 page 2: 逻辑数据页 1  
...
```

RID 里的 **pageNum** 是逻辑数据页号，不是物理页号。

所以：

```
RID(pageNum = 0, slotNum = 3)
```

表示第一张数据页的第 3 个 slot。

RecordManager

文件：

```
engine-project-master/src/main/java/edu/sustech/cs307/system/RecordManager.java
```

CreateFile(String filename, int record_size)

创建 record 文件。

初始化文件头：

- record size

- number of pages
- first free page
- records per page
- bitmap size

注意：

```
recordFileHeader.setNumberOfPages(1);
```

表示刚创建时只有文件头页，还没有数据页。

OpenFile(String table_name)

打开表的数据文件，读取 page 0 上的 RecordFileHeader。

注意参数传入的是表名，内部拼成：

```
table_name/data
```

CloseFile(RecordFileHandle recordFileHandle)

flush 这个文件的所有页。

RecordFileHeader

文件头结构。

包含：

- record size
- number of pages
- records per page
- first free page
- bitmap size

RecordPageHeader

每个数据页的页头。

包含：

- next free page number
- number of records

RecordPageHandle

表示一个数据页的操作句柄。

关键方法：

`getSlot(int slotNo)`

根据 slot 编号返回这一条记录所在的 `ByteBuffer` 切片。

页内布局大概是：

```
RecordPageHeader
Bitmap
slot 0
slot 1
slot 2
...
```

`RecordFileHandle`

记录层最核心的类。

`FetchPageHandle(int pageId)`

根据逻辑数据页号取数据页。

内部映射：

```
dataPagePosition(pageId)
-> (pageId + 1) * Page.DEFAULT_PAGE_SIZE
```

所以逻辑页 0 读物理偏移 4096。

`InsertRecord(ByteBuf buf)`

插入一条记录。

流程：

1. 调用 `create_page_handle()` 找到有空位的数据页；没有就新建页。
2. 用 bitmap 找第一个空 slot。
3. 把 record bytes 写入 slot。
4. bitmap 标记该 slot 已使用。
5. 页记录数加 1。
6. 如果页满了，就把文件头里的 first free page 指向下一个空闲页。
7. 返回 RID。

`DeleteRecord(RID rid)`

删除记录。

流程：

1. 根据 RID 找数据页。
2. 判断页删除前是否是满页。
3. bitmap 清掉 slot。
4. 页记录数减 1。
5. 如果这个页原来是满的，删除后重新加入 free page 链表。

UpdateRecord(RID rid, ByteBuf buf)

根据 RID 找到 slot，然后用新 bytes 覆盖。

CreateNewPageHandle()

创建新的数据页。

流程：

1. 调用 `bufferPool.NewPage(filename)`。
2. 初始化 bitmap。
3. 初始化 page header。
4. `numberOfPages + 1`。
5. 设置 first free page。

RID

记录 ID：

```
int pageNum;  
int slotNum;
```

用于唯一定位一条记录。

也实现了 `equals` 和 `hashCode`，方便索引里删除 RID。

BitMap

用于记录页内 slot 是否被占用。

常见方法：

- `init`
- `isSet`
- `set`
- `reset`
- `firstBit`

答辩说法：

Record 层把一张表看成一个 record 文件。文件第一页是 header，后面每页是数据页。每个数据页用 bitmap 记录 slot 是否占用，RID 用逻辑页号和 slot 号定位记录。

12. 存储层：storage

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/storage
```

Page

表示内存中的一个页。

字段：

- data
- position
- dirty
- pin_count

页大小是：

```
Page.DEFAULT_PAGE_SIZE
```

PagePosition

唯一定位一个磁盘页。

包含：

- filename
- offset

例如：

```
t/data, offset 4096
```

DiskManager

负责真实文件 I/O。

CreateFile(String filename)

创建磁盘文件，并初始化 filePages。

ReadPage(Page page, String filename, int offset, long length)

从磁盘文件指定 offset 读一页到内存 Page。

FlushPage(Page page)

把内存页写回磁盘。

AllocatePage(String filename)

给文件分配新页号。

DeleteFile(String filename)

删除磁盘文件。

dump_disk_manager_meta(...) / read_disk_manager_meta()

保存和读取文件页数元数据。

BufferPool

缓冲池。

数据库不会每次都直接读磁盘，而是先从 buffer pool 找页。

关键字段：

- `pages`：内存 frame 数组。
- `pageMap`：PagePosition -> frame id。
- `freeList`：空闲 frame。
- `replacer`：LRU 或 Clock 替换器。

关键方法：

FetchPage(PagePosition position)

取一页。

流程：

1. 如果 pageMap 里已有，说明页在内存，pin_count 加 1。
2. 如果不在内存，找 victim frame。
3. victim 如果 dirty，先 flush。
4. 从磁盘读新页到 frame。
5. pin_count 加 1。

NewPage(String filename)

为某个文件分配新页，并放进 buffer pool。

unpin_page(PagePosition position, boolean is_dirty)

使用完一页后取消 pin。

如果 pin_count 变为 0，就交给 replacer 管理。

FlushPage(PagePosition position)

刷单页。

FlushAllPages(String filename)

刷某个文件或所有文件的页。

DeleteAllPages(String filename)

从 buffer pool 移除某个文件的所有页。

ClearCache()

清空缓冲池。

事务 rollback 和 ALTER 重建文件后会用到。

答辩说法：

BufferPool 是磁盘和上层 record 之间的缓存。pin_count 表示页正在被使用，不能被替换；dirty 表示页被修改过，替换前要写回磁盘。

13. 页面替换：storage.replacer

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/storage/replacer
```

PageReplacer

接口。

方法：

```
int Victim();  
void Pin(int frameId);  
void Unpin(int frameId);  
void Reset();
```

LRUReplacer

实现 Least Recently Used。

核心思想：

- unpin 的 frame 才能被替换。
- 最近使用过的 frame 放到队尾。
- victim 时取最久未使用的 frame。
- pin 后从可替换集合移除。

答辩说法：

LRU 的 victim 是“最久没有被访问且当前不被 pin 的 frame”。

ClockReplacer

实现 Clock / Second Chance。

核心思想：

- 每个 frame 有 ref bit。
- victim 指针像时钟一样循环。
- 如果 ref bit 是 1，就清 0 给第二次机会。
- 如果 ref bit 是 0，就选为 victim。
- pin 后不能替换。
- unpin 后加入候选集合并设置引用位。

答辩说法：

Clock 是 LRU 的近似实现，用一个环形指针和引用位减少维护成本。

14. 值系统：value

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/value
```

ValueType

内部类型：

```
INTEGER  
FLOAT  
CHAR
```

SQL 类型映射：

```
int/integer -> INTEGER
float/double -> FLOAT
char/varchar -> CHAR
```

Value

封装一个值。

关键字段：

```
Object value;
ValueType type;
```

固定长度：

```
INT_SIZE = 8
FLOAT_SIZE = 8
CHAR_SIZE = 64
```

关键方法：

ToByte()

把值序列化成字节。

FromByte(byte[] bytes, ValueType type)

从字节反序列化成值。

ValueComparer

负责比较两个 **Value**。

关键方法：

```
compare(Value v1, Value v2)
```

返回：

```
v1 > v2 -> 1
v1 = v2 -> 0
v1 < v2 -> -1
```

如果类型不同，抛出 `DBException`。

15. 索引：index

包：

```
engine-project-master/src/main/java/edu/sustech/cs307/index
```

Index

索引接口。

定义：

- `EqualTo`
- `LessThan`
- `MoreThan`
- `Range`

BPlusTreeIndex

内存 B+ Tree 实现。

核心结构：

```
Node
├─ InternalNode: keys + children
└─ LeafNode: keys + List<RID> values + next leaf pointer
```

关键字段：

```
private static final int MAX_KEYS = 4;
private Node root = new LeafNode();
private LeafNode firstLeaf = (LeafNode) root;
```

`insert(Value key, RID rid)`

插入索引项。

流程：

1. `findLeaf(key)` 找到叶子节点。
2. `lowerBound` 找插入位置。
3. 如果 key 已存在，把 RID 放进 bucket。
4. 如果 key 不存在，插入 key 和 RID bucket。
5. 如果叶子 key 数超过 `MAX_KEYS`，调用 `splitLeaf`。

`delete(Value key, RID rid)`

删除索引项。

流程：

1. 找叶子。
2. 找 key。
3. 从 key 对应的 RID bucket 删除 RID。
4. bucket 空了就删除 key。
5. 调用 `collapseEmptyRoot()` 简单收缩根。

`equalToRids(Value key)`

返回等于 key 的所有 RID。

`lessThanRids(Value key, boolean inclusive)`

从 `firstLeaf` 开始顺着叶子链扫描，返回小于 key 的 RID。

`moreThanRids(Value key, boolean inclusive)`

从 key 所在叶子开始，顺着叶子链扫描大于 key 的 RID。

`rangeRids(...)`

范围查询。

`printTree()`

打印树结构。

输出类似：

```
Internal[30]
  Internal[24, 27]
    Leaf[21, 22, 23]
    Leaf[24, 25, 26]
```

`splitLeaf(LeafNode leaf)`

叶子节点分裂。

分裂后：

- 左叶子保留前半部分。
- 右叶子拿后半部分。
- 维护 `next` 链表。
- 把右叶子的第一个 key 插入父节点。

splitInternal(InternalNode internal)

内部节点分裂。

把中间 key 提升到父节点。

insertIntoParent(...)

把分裂产生的新节点插入父节点。

如果原来没有父节点，就创建新 root。

答辩说法：

B+ Tree 内部节点只保存 key 和 child 指针，真正的 RID 存在叶子节点；叶子节点之间用 next 指针串起来，方便范围查询。

InMemoryOrderedIndex

目前继承 `BPlusTreeIndex`，保留原接口兼容。

16. 事务：system.TransactionManager

文件：

```
engine-project-  
master/src/main/java/edu/sustech/cs307/system/TransactionManager.java
```

这个事务实现是快照式事务。

核心字段：

```
private Path transactionSnapshot;  
private final List<SavepointSnapshot> savepoints;
```

begin()

开始事务。

流程：

1. 如果已有活跃事务，抛异常。
2. 调用 `createSnapshot()` 复制整个数据库目录。
3. 清空 savepoints。

答辩说法：

begin 时做的是物理目录快照，把当前数据库根目录复制到临时目录。

commit()

提交事务。

流程：

1. 如果没有活跃事务，直接返回。
2. 调用 `dbManager.persistRuntimeState()` 把当前状态写盘。
3. 删除 transaction snapshot 和所有 savepoint snapshot。

答辩说法：

commit 不需要复制回来，因为当前数据库目录已经是提交后的状态；只需要持久化并清理快照。

rollback()

回滚整个事务。

流程：

1. 如果没有活跃事务，直接返回。
2. 调用 `restoreSnapshot(transactionSnapshot)`。
3. 清理事务状态。

savepoint(String savepointName)

创建保存点。

流程：

1. 要求当前有事务。
2. 复制当前数据库目录。
3. 保存 {name, snapshot path}。

rollbackToSavepoint(String savepointName)

回滚到最近的同名 savepoint。

流程：

1. 找最新的 savepoint。
2. 用它的 snapshot 恢复数据库目录。
3. 删除它之后创建的 savepoint。

releaseSavepoint(String savepointName)

删除一个保存点 snapshot。

restoreSnapshot(Path snapshot)

事务恢复核心方法。

流程：

1. 删除当前数据库目录内容。
2. 把 snapshot 复制回数据库目录。
3. 清空 buffer pool。
4. reload disk manager metadata。
5. reload table metadata JSON。
6. 清空 runtime index，下次使用时重建。

答辩说法：

事务使用目录级 snapshot 实现，简单但可靠。rollback 就是把 begin 或 savepoint 时的目录复制回来，并清理内存缓存和运行时索引。

17. 高级 SQL 和普通 SQL 的区别

普通 SQL：

```
select t.id from t where t.age > 18;
```

执行方式：

```
SeqScan -> Filter -> Project
```

它可以流式执行，每次只处理一行。

高级 SQL：

```
select t.age, count(*), max(t.gpa)
from t
group by t.age
order by t.age;
```

执行方式：

```
AdvancedSelectOperator
-> 先 materialize 所有相关行
-> where
-> group
-> aggregate
-> order
-> output
```

为什么要物化？

- GROUP BY 需要看到同一组的所有行。

- `ORDER BY` 需要比较多行顺序。
- `EXISTS` / `IN` 子查询可能需要反复执行子查询。

所以这部分没有走普通 `ProjectOperator`，而是单独的 `AdvancedSelectOperator`。

18. SQL 示例和对应执行链路

示例 1：建表

```
create table t(id int, name varchar, age int, gpa double);
```

链路：

```
LogicalPlanner.resolveAndPlan  
-> JSqlParser.parse CreateTable  
-> CreateTableExecutor.execute  
-> DBManager.createTable  
-> MetaManager.createTable  
-> RecordManager.CreateFile  
-> DiskManager.CreateFile
```

示例 2：插入

```
insert into t(id, name, age, gpa) values (1, 'a', 18, 3.6);
```

链路：

```
LogicalPlanner.handleInsert  
-> LogicalInsertOperator  
-> PhysicalPlanner.handleInsert  
-> InsertOperator.Begin  
-> RecordFileHandle.InsertRecord  
-> DBManager.addRecordToIndexes
```

示例 3：普通查询

```
select t.id, t.name from t where t.age > 18;
```

链路：

```
LogicalPlanner.handleSelect  
-> LogicalTableScanOperator  
-> LogicalFilterOperator  
-> LogicalProjectOperator  
-> PhysicalPlanner.generateOperator  
-> SeqScanOperator  
-> FilterOperator  
-> ProjectOperator  
-> DBEntry 打印结果
```

示例 4: COUNT

```
select count(*) from t where t.age > 18;
```

链路:

```
LogicalTableScanOperator  
-> LogicalFilterOperator  
-> LogicalCountOperator  
-> SeqScanOperator  
-> FilterOperator  
-> CountOperator
```

示例 5: Join

```
select t.id, d.title  
from t join d on t.id = d.student_id  
where d.student_id <= 8;
```

链路:

```
LogicalTableScanOperator(t)  
-> LogicalJoinOperator(TableScan(d), on t.id = d.student_id)  
-> LogicalFilterOperator(d.student_id <= 8)  
-> LogicalProjectOperator  
-> NestedLoopJoinOperator  
-> FilterOperator  
-> ProjectOperator
```

示例 6: GROUP BY / ORDER BY

```
select t.age, count(*), max(t.gpa)
from t
group by t.age
order by t.age;
```

链路：

```
LogicalPlanner.requiresAdvancedSelect == true
-> LogicalAdvancedSelectOperator
-> PhysicalPlanner
-> AdvancedSelectOperator
```

示例 7：索引查询

```
create index idx_age on t(age);
select t.id from t where t.age >= 20;
```

创建索引链路：

```
LogicalPlanner.handleManualDdlCommand
-> DBManager.createIndex
-> TableMeta.addIndex
-> MetaManager.saveToJson
-> DBManager.rebuildIndex
-> SeqScanOperator 扫表
-> BPlusTreeIndex.insert
```

查询链路：

```
LogicalFilterOperator
-> PhysicalPlanner.handleFilter
-> findIndexedLookup
-> DBManager.lookupIndex
-> BPlusTreeIndex.moreThanRids
-> IndexScanOperator
-> FilterOperator 再校验完整条件
```

示例 8：事务

```
begin;
insert into t(id, name, age, gpa) values (100, 'tx', 20, 4.0);
```

```

savepoint s1;
insert into t(id, name, age, gpa) values (101, 'tx2', 21, 4.1);
rollback to savepoint s1;
commit;

```

链路:

```

begin
-> TransactionManager.begin
-> createSnapshot

savepoint s1
-> TransactionManager.savepoint
-> createSnapshot

rollback to savepoint s1
-> restoreSnapshot
-> reload metadata
-> clear buffer/cache/runtime indexes

commit
-> persistRuntimeState
-> clearTransactionState

```

19. Project2 任务对应实现位置

PDF 要求	实现位置
LRU	storage/replacer/LRUReplacer.java
Clock	storage/replacer/ClockReplacer.java
CREATE TABLE	logicalOperator/ddl/CreateTableExecutor.java, DBManager.createTable
INSERT	LogicalPlanner.handleInsert, PhysicalPlanner.handleInsert, InsertOperator
UPDATE	LogicalPlanner.handleUpdate, PhysicalPlanner.handleUpdate, UpdateOperator
DELETE	LogicalPlanner.handleDelete, PhysicalPlanner.handleDelete, DeleteOperator
SHOW TABLES	LogicalPlanner.handleManualDdlCommand, DBManager.showTables
DESC	LogicalPlanner.handleManualDdlCommand, DBManager.descTable
DROP TABLE	LogicalPlanner.handleManualDdlCommand, DBManager.dropTable
EXPLAIN	LogicalPlanner.explainSql, logical operators toString()

PDF 要求	实现位置
SELECT projection	<code>LogicalProjectOperator</code> , <code>ProjectOperator</code> , <code>ProjectTuple</code>
WHERE	<code>LogicalFilterOperator</code> , <code>FilterOperator</code> , <code>Tuple.eval_expr</code>
SeqScan	<code>SeqScanOperator</code> , <code>RecordFileHandle.FetchPageHandle</code>
COUNT	<code>LogicalCountOperator</code> , <code>CountOperator</code>
JOIN	<code>LogicalJoinOperator</code> , <code>NestedLoopJoinOperator</code> , <code>JoinTuple</code>
MAX/MIN/GROUP/ORDER	<code>LogicalAdvancedSelectOperator</code> , <code>AdvancedSelectOperator</code>
IN/NOT IN/EXISTS	<code>AdvancedSelectOperator.evaluateCondition</code> , <code>evaluateInExpression</code> , <code>executeSelectExpression</code>
ALTER TABLE	<code>LogicalPlanner.handleManualDdlCommand</code> , <code>DBManager.alterTableAddColumn</code> , <code>alterTableDropColumn</code>
CREATE/DROP INDEX	<code>LogicalPlanner.handleManualDdlCommand</code> , <code>DBManager.createIndex</code> , <code>dropIndex</code>
B+ Tree	<code>index/BPlusTreeIndex.java</code>
索引扫描	<code>PhysicalPlanner.handleFilter</code> , <code>IndexScanOperator</code>
事务	<code>TransactionManager.java</code>
CLI 展示	<code>DBEntry.java</code>

20. 推荐答辩讲法

如果老师让你介绍系统，可以按这个顺序讲：

1. 先说整体架构：

```
SQL -> LogicalPlanner -> PhysicalPlanner -> PhysicalOperator -> Tuple ->
Record/Storage
```

2. 再讲普通查询：

```
SeqScan 负责扫表
Filter 负责 where
Project 负责 select 列
```

3. 再讲存储：

```
DiskManager 管文件 I/O
BufferPool 管页缓存
```

`RecordFileHandle` 管记录页和 RID
`Bitmap` 管 slot 是否占用

4. 再讲索引：

`create index` 记录元数据并扫表建 B+ Tree
`insert/delete/update` 动态维护 B+ Tree
`where` 条件能用索引时走 `IndexScanOperator`

5. 再讲事务：

`begin/savepoint` 做目录快照
`rollback` 恢复快照
`commit` 持久化并删除快照

6. 最后讲高级 SQL：

`GROUP BY / ORDER BY / IN / EXISTS` 走 `AdvancedSelectOperator`
它采用物化执行，先读行，再过滤、分组、聚合、排序

21. 常见老师问题和回答思路

Q1: `SeqScanOperator` 怎么知道哪些 slot 有记录？

回答：

每个数据页有 bitmap。`SeqScanOperator.hasNext()` 会拿到当前页的 `RecordPageHandle`，然后遍历 slot，用 `Bitmap.isSet(...)` 判断该 slot 是否有效。如果有效，就在 `Next()` 里根据当前 page/slot 构造 RID，然后调用 `RecordFileHandle.GetRecord(rid)` 取记录。

Q2: RID 里的 `pageNum` 是物理页号吗？

回答：

不是。RID 里的 `pageNum` 是逻辑数据页号，从 0 开始。物理 page 0 是文件头，所以逻辑数据页 0 对应物理 page 1。`RecordFileHandle.dataPagePosition(logicalPageId)` 里做了 $(\text{logicalPageId} + 1) * \text{PAGE_SIZE}$ 的映射。

Q3: WHERE 是在哪里执行的？

回答：

普通 where 在 `FilterOperator` 中执行，它会调用 tuple 的 `eval_expr`。`Tuple.eval_expr` 支持 AND、OR 和二元比较。高级 where 里的 IN/EXISTS 则在 `AdvancedSelectOperator.evaluateCondition` 中执行。

Q4: 为什么高级 SELECT 单独做一个 `AdvancedSelectOperator`?

回答:

因为 GROUP BY、ORDER BY、聚合和子查询都需要看到多行甚至全部中间结果，流式 Project/Filter 不够方便。所以高级查询采用物化执行，把中间行读入 `MaterializedTuple` 列表后统一处理。

Q5: B+ Tree 为什么叶子节点要有 next 指针?

回答:

范围查询需要顺序访问叶子节点。B+ Tree 所有真实 RID 都存在叶子节点，叶子节点用 next 串起来后，`<`、`>`、range 查询可以从某个叶子开始向后扫描。

Q6: 索引什么时候更新?

回答:

`InsertOperator.Begin()` 插入记录后调用 `dbManager.addRecordToIndexes`。`DeleteOperator` 删除前调用 `removeRecordFromIndexes`。`UpdateOperator` 更新记录时调用 `updateRecordIndexes`，删除旧 key/RID，再插入新 key/RID。

Q7: 事务 rollback 怎么实现?

回答:

begin 时复制当前数据库目录作为 snapshot。rollback 时删除当前数据库目录内容，再把 snapshot 复制回来，然后清空 BufferPool，重新加载 DiskManager metadata 和 MetaManager JSON，并清空运行时索引。

Q8: commit 做了什么?

回答:

commit 时当前数据库目录已经是最新状态，所以只需要调用 `persistRuntimeState()` flush buffer 和保存元数据，然后删除事务 snapshot 和 savepoint snapshot。

Q9: 为什么 ALTER TABLE 只支持空表?

回答:

因为 record 是固定长度二进制布局，非空表加列或删除列需要重写所有旧记录，否则旧 record 的长度和新 schema 不匹配。为了避免破坏数据，本实现只允许空表 ALTER，并重建空 data 文件。

Q10: 这个优化器算完整优化器吗?

回答:

不是完整成本优化器。它实现了课程项目中的简单 plan 选择：普通查询走 SeqScan，简单单表索引条件可以走 IndexScan，高级查询走 `AdvancedSelectOperator`。

22. 建议现场演示 SQL

可以按这个顺序演示：

```
create table t(id int, name varchar, age int, gpa double);
create table d(student_id int, title varchar);
```

插入 30 行以上数据。

基础查询：

```
show tables;
desc t;
select * from t;
select t.id, t.name from t where t.age >= 20 and t.id <= 10;
select count(*) from t where t.age > 18;
```

Join：

```
select t.id, d.title from t join d on t.id = d.student_id;
```

高级查询：

```
select t.age, count(*), max(t.gpa), min(t.id)
from t
group by t.age
order by t.age;
```

子查询：

```
select t.id from t
where t.id in (select d.student_id from d)
order by t.id;
```

相关 exists：

```
select t.id from t
where exists (select d.student_id from d where d.student_id = t.id)
order by t.id;
```

索引：

```
create index idx_age on t(age);
print index idx_age;
select t.id from t where t.age >= 20;
drop index idx_age;
```

事务：

```
begin;
insert into t(id, name, age, gpa) values (100, 'tx100', 23, 4.0);
savepoint s1;
insert into t(id, name, age, gpa) values (101, 'tx101', 24, 4.1);
rollback to savepoint s1;
commit;
select t.id from t where t.id >= 100;
```

空表 ALTER：

```
create table empty_t(id int);
alter table empty_t add column name varchar;
desc empty_t;
insert into empty_t(id, name) values (1, 'one');
select * from empty_t;
```

23. 你最应该先看哪些文件

如果时间很紧，按这个顺序看：

1. DBEntry.java
2. LogicalPlanner.java
3. PhysicalPlanner.java
4. PhysicalOperator.java
5. SeqScanOperator.java
6. FilterOperator.java
7. ProjectOperator.java
8. RecordFileHandle.java
9. BufferPool.java
10. DiskManager.java
11. DBManager.java
12. BPlusTreeIndex.java
13. TransactionManager.java
14. AdvancedSelectOperator.java

如果你只想能讲清楚 80 分基础部分，重点看 1 到 10。

如果你要讲加分项，继续看 11 到 14。

24. 当前测试文件对应功能

```
storage/LRUReplacerTest.java
```

验证 LRU 页面替换。

```
storage/ClockReplacerTest.java
```

验证 Clock 页面替换。

```
record/RecordFileHandleTest.java
```

验证记录插入、删除、更新、页分配、持久化。

```
system/RecordManagerTest.java
```

验证 record file 创建和 header。

```
system/Task2IntegrationTest.java
```

验证基础 SQL、30+ 数据、投影、where、delete、count、join、transaction demo 链路。

```
system/IndexIntegrationTest.java
```

验证 create/drop index、索引查询、insert/delete/update 维护索引、print index。

```
system/AdvancedQueryIntegrationTest.java
```

验证 MAX/MIN、GROUP BY、ORDER BY、IN/NOT IN、EXISTS/NOT EXISTS、空表 ALTER。

```
system/TransactionManagerTest.java
```

验证 rollback、savepoint、rollback to savepoint、release savepoint。

```
value/ValueComparerTest.java
```

验证不同类型 Value 的比较逻辑。

25. 最后记住这几句话

第一句：

这个项目的核心是把 SQL 变成逻辑计划，再变成物理算子，物理算子通过统一的 iterator 接口执行。

第二句：

数据最终存在 record 文件里，文件第一页是 header，后面是数据页，数据页用 bitmap 管理 slot。

第三句：

BufferPool 缓存磁盘页，pin_count 防止正在使用的页被替换，dirty 页在淘汰或 flush 时写回磁盘。

第四句：

普通查询走 SeqScan/Filter/Project，高级查询走 AdvancedSelectOperator，索引可用时走 IndexScan。

第五句：

事务用目录快照实现，rollback 就是恢复快照，commit 就是持久化并清理快照。